Министерство науки и высшего образования Российской Федерации
Пензенский государственный технологический университет
Кафедра «Программирование»

ОТЧЁТ

по лабораторной работе № 2
«Вывод модели в эксплуатацию»
по дисциплине «Прикладные задачи искусственного интеллекта»

Вариант 1. Отток клиентов телеком-оператора

Выполнил: студент группы 23ИВ1б
Терехин А.

Проверил: к.т.н., доцент
Маркин Е. И.

Пенза, 2026

1. Задача и требования варианта

Модель ЛР1 — пайплайн «сырые столбцы → признаки → LightGBM» (параметры Optuna, из новых признаков оставлен `n_services`), порог решения 0,10 по минимуму стоимости ошибок (пропуск уходящего 6000 ₽, скидка лояльному 800 ₽), ROC-AUC на тесте 0,849. Цель ЛР2 — превратить её в воспроизводимый сервис. Требования варианта 1: пакетный режим раз в сутки, **пакет 10 000 клиентов за 60 с**, требование к одиночному запросу не задано; сценарий дрейфа — рост доли помесячных договоров.

Вся работа — один ноутбук `lab2_terekhin.ipynb`: он записывает модули командой `%%writefile`, скачивает данные с проверкой MD5, обучает и регистрирует модель, поднимает сервис и строит все отчёты. Версии: scikit-learn 1.9.1 и LightGBM 4.7.0 (как в ЛР1), MLflow 3.17.2, FastAPI 0.161.1, Pydantic 2.14.1, uvicorn 0.54.0, Evidently 0.7.23, Locust 2.47.0, Gradio 6.30.0, pytest 9.1.1, pandas 3.0.5, numpy 2.5.3. Ветки из методички (mlflow 2, evidently 0.4, gradio 4, sklearn 1.5) не устанавливаются на Python 3.12+ и pandas 3, поэтому взяты актуальные версии; код Evidently написан под API 0.7, MLflow работает в файловом режиме через `MLFLOW_ALLOW_FILE_STORE=true`.

2. Схема решения

Модуль	Назначение	Использует
`src/features.py`	Признаки из ЛР1: `clean` (типы, пустой TotalCharges у новых клиентов), `add_features`, `CategoryAligner` (категории, запомненные на обучении); списки входных столбцов	—
`src/train.py`	Загрузка и MD5 данных, разбиение 80/20 (random_state 42), `train_model(X, y, params)` → пайплайн, CV, метрики, стоимость ошибок, схема входа	features
`src/predict.py`	`load_registered()` — модель из реестра MLflow по имени и псевдониму, порог из тегов версии; `predict_proba(список словарей)`	features, MLflow
`src/service.py`	FastAPI: схема Pydantic, 4 эндпоинта, загрузка модели один раз при старте (lifespan), журнал запросов	predict, monitoring
`src/monitoring.py`	Журнал запросов JSONL в фоновом потоке, PSI, суточный отчёт (доп. задание)	—
`tests/`, `locustfile.py`	19 проверок pytest; сценарий нагрузки	service

Поток данных: *данные → train.py → запуск MLflow (параметры, метрики, модель, схема входа) → реестр `lab2_terekhin@champion` → service.py (загрузка при старте) → клиенты: Gradio, Locust, тесты, суточный пакет → журнал запросов → суточный отчёт; Evidently сравнивает пакет с эталоном.* Модули при импорте ничего не выполняют, что проверяется импортом в отдельном процессе. Модель сохраняется со ссылкой на `src.features.add_features` и вместе с кодом пакета (`code_paths=['src']`), поэтому загружается в процессе сервиса и в процессе тестов. Обучение функцией из `src/train.py`

воспроизводит модель ЛР1 точно: максимальное расхождение вероятностей с `model.joblib` ЛР1 равно 0, ROC-AUC на тесте 0,8488.

3. Контракт сервиса

Эндпоинт	Вход	Выход
`GET /health`	—	`{"status": "ok", "model_loaded": true}`
`GET /model-info`	—	имя, версия и псевдоним в реестре, run_id, тип модели, порог, ROC-AUC теста, версия данных, список признаков, макс. размер пакета
`POST /predict`	объект Item	`customerID, proba, decision, threshold, model_version`
`POST /predict-batch`	список Item, 1…20 000	`n, threshold, model_version` и массивы `customerID, proba, decision`; пустой пакет — 400, больше 20 000 — 413

Схема входа Item (19 признаков, типы и границы — из `input_schema.json` ЛР1): категориальные поля — `Literal` с допустимыми значениями (например, `Contract ∈ {Month-to-month, One year, Two year}`); `SeniorCitizen ∈ {0, 1}`; `tenure` — целое 0…72; `MonthlyCharges` — 18…120; `TotalCharges` — 0…9000 или `null` (новый клиент); `customerID` — необязательная строка, в модель не передаётся. Лишние поля запрещены (`extra='forbid'`), проверяется согласованность услуг: `MultipleLines = 'No phone service'` тогда и только тогда, когда `PhoneService = 'No'`, и шесть интернет-услуг равны `'No internet service'` тогда и только тогда, когда `InternetService = 'No'`. Любое нарушение даёт 422 с указанием поля (для пакета — с номером объекта), а не 500.

```
POST /predict  {"customerID": "0001-TEST", "gender": "Female", "SeniorCitizen": 0,
"Partner": "Yes", "Dependents": "No",
 "tenure": 2, "PhoneService": "Yes", "MultipleLines": "No", "InternetService": "Fiber
optic", "OnlineSecurity": "No",
 "OnlineBackup": "No", "DeviceProtection": "No", "TechSupport": "No", "StreamingTV":
"Yes", "StreamingMovies": "No",
 "Contract": "Month-to-month", "PaperlessBilling": "Yes", "PaymentMethod":
"Electronic check",
 "MonthlyCharges": 85.5, "TotalCharges": 170.0}
→ 200 {"customerID": "0001-TEST", "proba": 0.7407, "decision": 1, "threshold": 0.1,
"model_version": 2}

тот же запрос с tenure = -5, Contract = "Weekly" и без MonthlyCharges
→ 422 {"detail": [
  {"type": "greater_than_equal", "loc": ["body", "tenure"], "msg": "Input should be
greater than or equal to 0", "input": -5},
  {"type": "literal_error", "loc": ["body", "Contract"], "msg": "Input should be
'Month-to-month', 'One year' or 'Two year'"},
  {"type": "missing", "loc": ["body", "MonthlyCharges"], "msg": "Field required"}]}

InternetService = "No" при подключённых интернет-услугах
→ 422 "Value error, OnlineSecurity = 'No internet service' тогда и только тогда,
когда InternetService = 'No'"
POST /predict-batch []  → 400 {"detail": "пустой пакет: нужен хотя бы один объект"}
```

4. Трекинг экспериментов и реестр моделей

Хранилище MLflow — каталог `mlruns/` в папке проекта на Диске, эксперимент `lab2`. Каждый запуск логирует параметры модели, метрики (ROC-AUC 5-кратной CV на

обучающей выборке, ROC-AUC, PR-AUC, Brier и стоимость ошибок на тесте, порог), теги `data_version` (MD5 данных), `model_type`, `session` (метка прогона ноутбука), артефакт `input_schema.json` и модель с сигнатурой и примером входа.

Таблица 1 — Запуски одного прогона ноутбука (второй прогон дал те же значения)

Запуск	**Модель**	**CV ROC-AUC**	**Тест ROC-AUC**	**PR-AUC**	**Стоимость, ₽/клиент**
lgbm-tuned	LightGBM, параметры Optuna ЛР1, новый признак n_services	**0,8511 ± 0,0119**	0,8488	0,665	382,4
lgbm-tuned-all-features	то же, все 7 новых признаков	0,8498 ± 0,0121	0,8480	0,665	381,0
logreg-baseline	логистическая регрессия	0,8461 ± 0,0126	0,8420	0,634	379,3
lgbm-default	LightGBM по умолчанию	0,8352 ± 0,0089	0,8357	0,640	422,1

Зарегистрирована модель `lab2_terekhin` из запуска **lgbm-tuned**: выбор сделан по ROC-AUC кросс-валидации, а не по тесту, чтобы не подгонять выбор под отложенную выборку; это та же модель, что выбрана в ЛР1. Версия получает теги `threshold = 0.1`, `data_version`, `roc_auc_test` и псевдоним `champion` (в MLflow 3 псевдонимы заменили стадии Staging/Production). Сервис загружает модель как `models:/lab2_terekhin@champion`, а не по пути к файлу; ноутбук дополнительно проверяет загрузку по имени и номеру версии. Каждый прогон ноутбука добавляет новую версию — после двух прогонов в реестре версии 1 и 2, `champion` указывает на 2 (рис. 1).

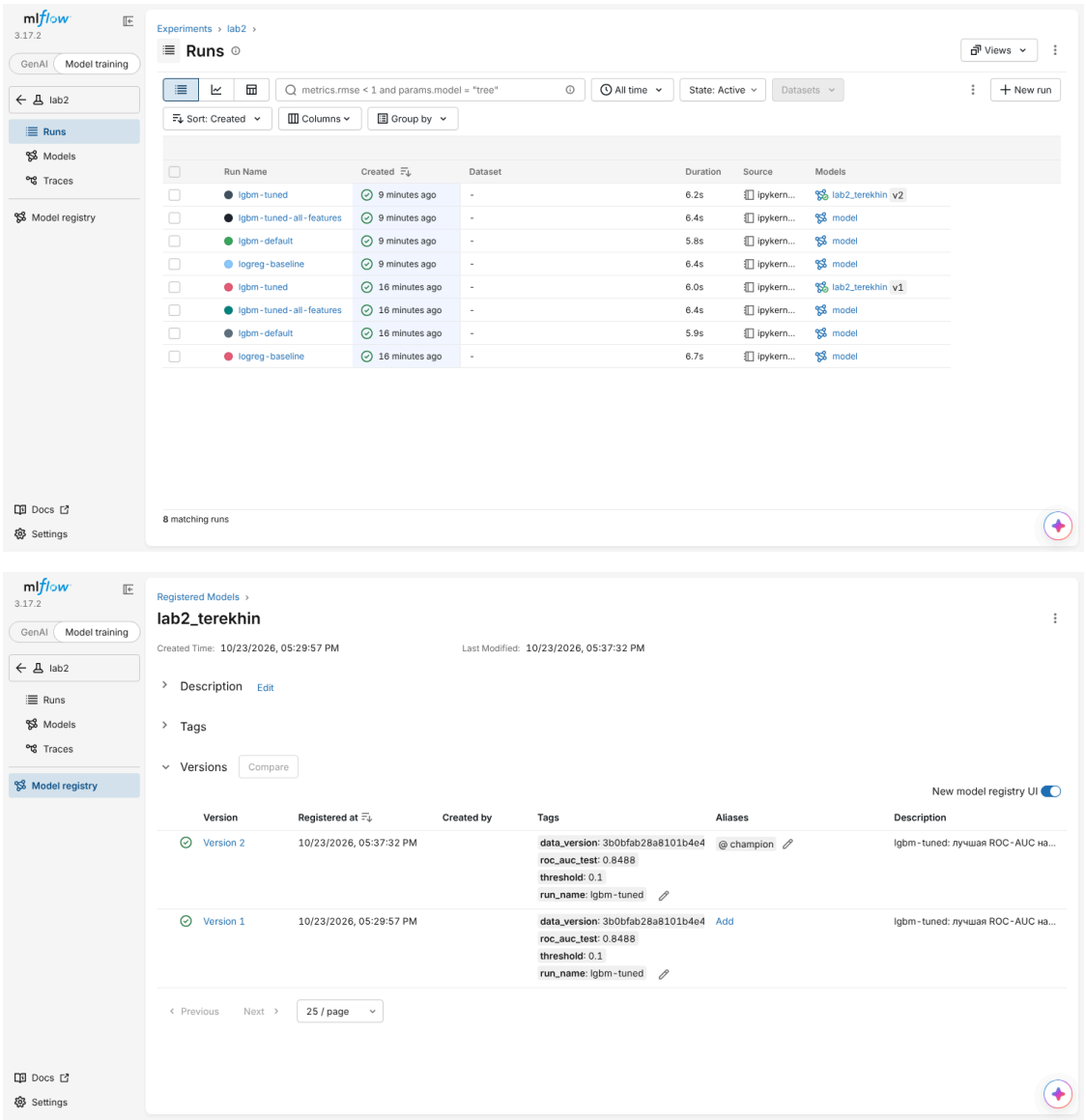


Рисунок 1 — Запуски эксперимента lab2 (два прогона ноутбука) и реестр модели lab2_terekhin в MLflow UI

5. Тесты

Тесты используют `TestClient` в контексте `with`, поэтому выполняется lifespan и модель загружается из реестра так же, как в рабочем сервисе. Журнал тестовых запросов пишется в отдельный файл.

Тест	Что проверяет
test_health, test_model_info	200 и `status = ok`; имя, версия ≥ 1, порог 0,10, 19 признаков без customerID
test_predict_ok	200, вероятность в [0, 1], `decision = [proba ≥ threshold]`, возврат customerID
test_risk_ordering	новый клиент с помесячным договором рискованнее клиента с 60 мес. стажа и двухлетним договором
test_new_client_without_total_charges	`tenure = 0, TotalCharges = null` → 200 (ловушка данных из ЛР1)
test_wrong_type, test_missing_field	`tenure = "много"` и отсутствие `Contract` → 422 с именем поля
test_out_of_range (6 случаев)	tenure −1 и 500, MonthlyCharges 1000, TotalCharges −5, SeniorCitizen 2, Contract «Weekly» → 422
test_unknown_field, test_inconsistent_services	лишнее поле → 422; интернет-услуги без интернета → 422 с понятным сообщением
test_batch_1000	пакет из 1000: 200, длины массивов 1000, вероятности в [0, 1], решения согласованы с порогом, ответ совпадает с одиночным
test_batch_empty, test_batch_too_large, test_batch_with_bad_item	пустой пакет → 400; 20 001 объект → 413; ошибка во 2-м объекте → 422 с `loc = [body, 1, tenure]`

Результат: `19 passed in 4.11s` (файлы `reports/pytest.txt`, `reports/pytest.xml`).

6. Нагрузочный тест

Условия измерения. Сервер uvicorn — один процесс в фоновом потоке ядра Jupyter; Locust — отдельный процесс на той же машине, безголовый режим, по 60 с на уровень нагрузки, ожидание пользователя 10–50 мс. Сценарий: одиночный запрос (вес 5) и пакет из 100 клиентов (вес 1); клиенты — реальные записи тестовой выборки. Машина — 4 ядра CPU, Linux, Python 3.13; в Colab (2 vCPU, разделяемая среда) абсолютные значения будут выше и шумнее, поэтому выводы делаются о соотношении режимов.

Таблица 2 — Задержки и пропускная способность (Locust)

Пользователей	Эндпоинт	Запросов	Ошибок	p50, мс	p95, мс	p99, мс	Запросов/с
1	/predict	597	0 %	47	70	84	10,1
1	/predict-batch (100)	120	0 %	51	70	76	2,0
5	/predict	540	0 %	430	540	590	9,2
5	/predict-batch (100)	92	0 %	440	580	720	1,6
20	/predict	533	0 %	1800	2200	2400	9,0
20	/predict-batch (100)	105	0 %	1800	2100	2300	1,8
50	/predict	529	0 %	4400	5600	6800	9,0
50	/predict-batch (100)	108	0 %	4300	5300	5800	1,8

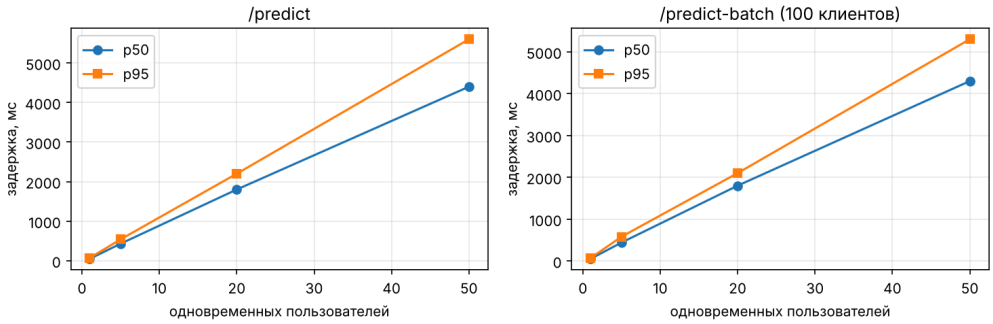


Рисунок 2 — p50 и p95 в зависимости от числа одновременных пользователей

Предел одного процесса — около 11 запросов/с суммарно (≈ 9 одиночных и 2 пакетных), и он достигается уже при 5 пользователях: дальше пропускная способность не растёт, а задержка растёт почти линейно с числом пользователей (очередь к одному процессу; ошибок нет ни на одном уровне). Если бы для одиночного запроса действовало требование, например, p95 ≤ 300 мс, оно выполнялось бы только при 1 пользователе. Профилирование показало, что ~33 из ~42 мс одиночного запроса — подготовка признаков в pandas (накладные расходы на каждый столбец), а не LightGBM (~7 мс); эти расходы почти не зависят от числа строк, поэтому пакет из 100 клиентов обрабатывается так же быстро, как один клиент.

Таблица 3 — Одиночный и пакетный режимы (последовательные запросы с одного клиента)

Режим	**Время, с**	**Клиентов/с**	**Задержка запроса**
1000 одиночных запросов подряд	48,6	21	p50 45 мс, p95 65 мс
1 пакет × 1000 клиентов (медиана 5 повторов)	0,090	11 100	90 мс
1 пакет × 10 000 клиентов (медиана / худший из 5)	**0,50 / 0,80**	≈ 20 000	501 / 801 мс

Вывод. Пакетный режим быстрее одиночных запросов в ≈ 540 раз: разбор запроса, валидация и вызов пайплайна распределяются на весь пакет. Суточный пакет варианта (10 000 клиентов) обрабатывается одним запросом менее чем за 1 с даже в худшем повторе — требование «за 60 с» выполнено с запасом ≈ 75 раз, так что запас есть и для более медленной среды Colab. Одиночный эндпоинт нужен для тестов, интерфейса и единичных проверок; для онлайн-нагрузки его следовало бы масштабировать несколькими процессами uvicorn (`--workers`).

7. Дрейф данных и правило переобучения

Сценарий. Оператор запускает помесячный тариф без обязательств, и в суточный пакет попадает больше клиентов с договором Month-to-month. Эталон — тестовая выборка (1409 клиентов, доля помесячных 54,9 %). «Новые» данные строятся перевзвешенной выборкой реальных клиентов эталона: строки с помесячным договором выбираются чаще, пока их доля не станет заданной (60–85 %). Это не шум: каждая строка остаётся реальным клиентом, а вместе с `Contract` естественным образом смещаются сопряжённые признаки (у помесячных клиентов меньше стаж и сумма платежей). Разметка известна, поэтому видно и изменение качества. Проверка — Evidently `DataDriftPreset` по 19 входным признакам с методами по умолчанию (при > 1000 строк — нормированное расстояние Вассерштейна для числовых и расстояние Йенсена — Шеннона для категориальных, порог 0,1).

Основной сценарий (доля помесячных 75 %, `reports/drift_report.html`, рис. 3): сдвиг обнаружен в 3 из 19 признаков (доля 0,158) — `tenure` (0,232), `TotalCharges` (0,154), `Contract` (0,139); следующий по величине — `PaymentMethod` (0,067). Встроенный порог Evidently «датасет сдвинут при доле ≥ 0,5» сдвиг не замечает: изменение сосредоточено в нескольких связанных признаках.

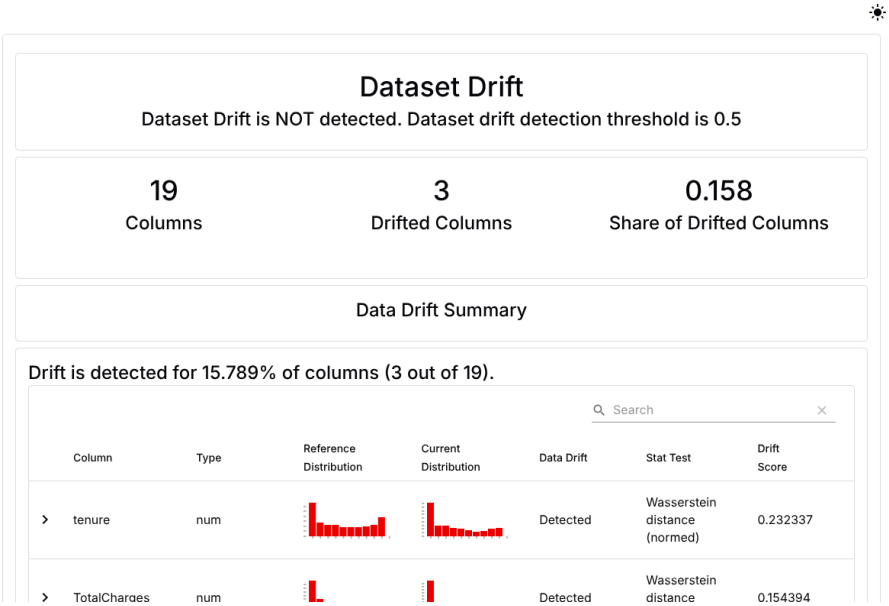


Рисунок 3 — Отчёт Evidently для сценария «75 % помесячных договоров»

Таблица 4 — Сценарии разной силы, по 10 случайных выборок на сценарий (средние значения)

Сценарий	Доля дрейфа (мин–макс)	Contract сдвинут	PSI прогнозов	Доля ушедших	ROC-AUC	Стоимость, ₽/кл.	Срабатывание правила
контроль (55 %)	0,000 (0–0)	0 %	0,007	0,266	0,848	385	0 %
M2M 60 %	0,000 (0–0)	0 %	0,013	0,284	0,841	395	0 %
M2M 65 %	0,058 (0–0,105)	0 %	0,037	0,299	0,831	411	0 %
M2M 70 %	0,142 (0,053–0,158)	80 %	0,066	0,319	0,829	407	80 %
M2M 75 %	0,158 (0,158–0,158)	100 %	0,124	0,338	0,816	414	100 %
M2M 85 %	0,216 (0,158–0,316)	100 %	0,342	0,377	0,788	430	100 %

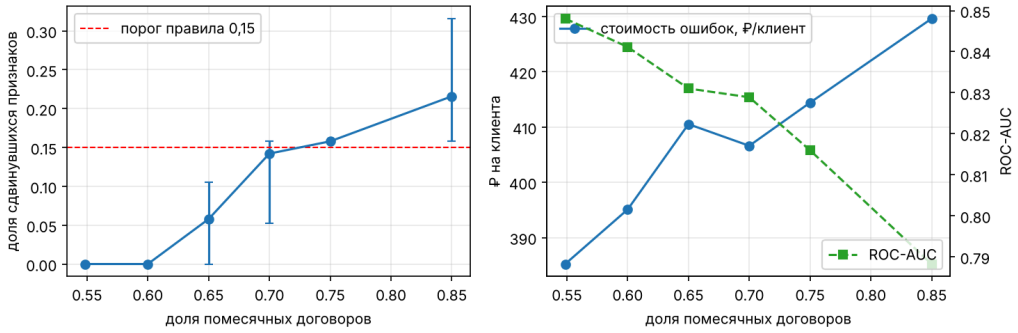


Рисунок 4 — Доля сдвинувшихся признаков и качество модели в зависимости от доли помесячных договоров

С ростом доли помесячных договоров растёт доля уходящих (с 26,6 до 37,7 %) и доля клиентов, которым модель предлагает скидку; ROC-AUC падает с 0,848 до 0,788 — на смещённой выборке больше трудноразличимых клиентов, а стоимость ошибок на клиента растёт на 12 %. Отдельные признаки при этом остаются в своих диапазонах, сервис отвечает штатно, ошибок в журнале нет — сдвиг виден только по распределениям. Это дрейф данных: зависимость оттока от признаков в сценарии не меняется (метки берутся у тех же клиентов), поэтому его можно обнаружить сразу по входам, не дожидаясь фактических уходов.

Правило переобучения (реализовано функциями `day_alarm` и `retrain_decision` в ноутбуке):

1. *Ежедневно* после ночного пакетного скоринга суточный пакет сравнивается с эталоном отчётом Evidently `DataDriftPreset`. Срабатывание дня: доля сдвинувшихся признаков ≥ **0,15** (не менее 3 из 19) **или** PSI распределения прогнозов относительно эталона ≥ **0,1**.
2. *Ежемесячно*, когда становится известен факт ухода клиентов, оценённых месяц назад, на выборке не менее 1000 размеченных клиентов: ROC-AUC < **0,819** (приёмочный уровень 0,849 − 0,03) **или** стоимость ошибок > **459 ₽** на клиента (1,2 × эталонной 382 ₽).
3. *Действие*: если условие 1 выполнено **3 дня подряд** либо выполнено условие 2 — переобучение того же пайплайна (`src/train.py`, параметры ЛР1, порог заново по минимуму стоимости на OOF) на данных за последние 12 месяцев; новая версия регистрируется с псевдонимом `challenger` и получает `champion`, только если на отложенном последнем месяце её стоимость ошибок ниже, а ROC-AUC не ниже, чем у текущей. После замены эталон обновляется.

Пороги выбраны по моделированию (табл. 4), а не взяты из примера методички (30 %): сдвиг сосредоточен в 3 сопряжённых признаках, и даже при 85 % помесячных договоров доля сдвинувшихся признаков в среднем лишь 0,22 — порог 0,3 пропустил бы и сильный сдвиг. Порог 0,15 ни разу не срабатывает в контроле и при 60–65 %, срабатывает в 80 % дней при 70 % и всегда — при 75 %, т.е. там, где ROC-AUC опускается ниже допуска 0,819. Требование «3 дня подряд» отсекает единичный необычный пакет. Проверка в ноутбуке: три суточных пакета со сдвигом 75 % → «переобучить: дрейф данных 3 дня подряд»; три дня без сдвига → «не требуется».

8. Веб-интерфейс и доп. задание

Gradio. Интерфейс из 19 полей (выпадающие списки для категорий, числа для стажа и платежей) вызывает сервис по HTTP (`POST /predict`), а не модель напрямую, и показывает вероятность, решение, порог и версию модели; ошибки 422 выводит списком полей. В Colab `launch(share=True)` выдаёт публичную ссылку `*.gradio.live` для демонстрации на защите.

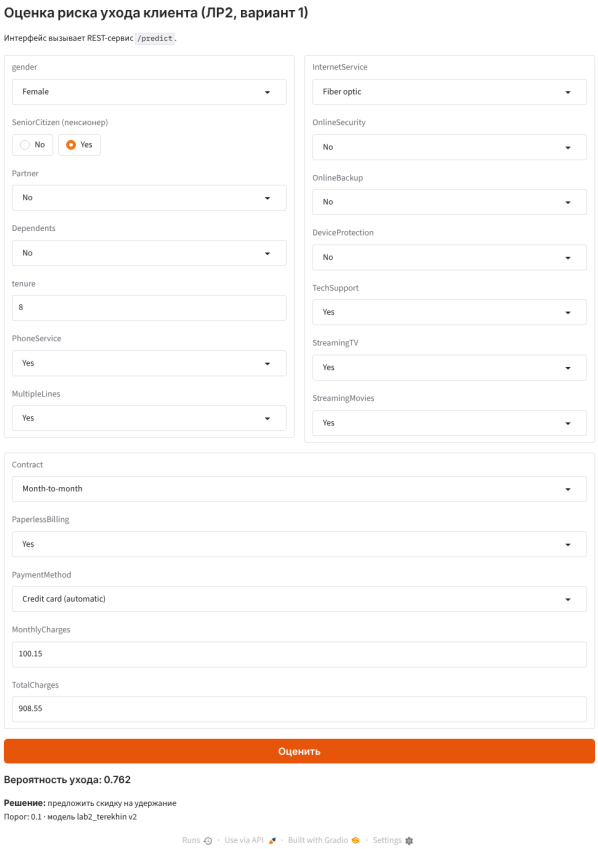


Рисунок 5 — Интерфейс Gradio: клиент с помесячным договором, вероятность ухода 0,762, решение — предложить скидку

Доп. задание — журнал запросов и суточный отчёт. Сервис записывает каждый запрос в `logs/predictions.jsonl`: время, эндпоинт, код ответа (в том числе 422 — через обработчик ошибок валидации), идентификатор запроса, число объектов, задержку, версию модели, порог и вероятности. Запись идёт через очередь в фоновом потоке, поэтому не добавляется к задержке ответа. Функция `daily_report` строит по журналу отчёт за сутки (`reports/daily_report.html`): число запросов и отказов, задержки по эндпоинтам, средняя вероятность и доля клиентов со скидкой против эталона, квантили, гистограмма и PSI распределения прогнозов — за сутки и по каждому пакету. В ноутбуке журнал нагрузочного теста ротируется, затем моделируются сутки: суточный пакет (1409 клиентов), 20 одиночных запросов, один ошибочный и дополнительный пакет со сдвигом 75 %. Отчёт показывает: первый пакет — PSI 0,000 («стабильно»), второй — PSI 0,155 («умеренный сдвиг»), средняя вероятность 0,343 против 0,266, доля клиентов со скидкой 0,80 против 0,63. Этот PSI и есть второй сигнал ежедневной проверки в правиле переобучения; журнал заодно даёт данные для будущего переобучения.

9. Воспроизводимость

Зафиксированы код (модули пишутся из ноутбука, репозиторий Git), данные (MD5 `3b0bfab2…a5c235` проверяется при загрузке, тег `data_version` в каждом запуске), окружение (версии в ячейке установки и `pip freeze` в `requirements.txt`) и случайные зёрна (`random_state = 42` в разбиении, моделях и CV; сценарии дрейфа с фиксированными seed). Если установка пакетов обновит уже загруженный в Colab numpy или pandas, ноутбук сам перезапускает среду выполнения — после этого снова «Выполнить всё».

Проверка: ноутбук выполнен целиком сверху вниз дважды подряд, начиная с пустого каталога проекта (без `mlruns/`, отчётов и журналов), в новом ядре, без ручных действий, путь проекта — как на Google Диске в Colab (`/content/drive/MyDrive/lab2_terekhin`). Оба прогона прошли без ошибок (≈ 7,5 мин, из них 4 мин — нагрузочный тест); метрики одноимённых запусков двух прогонов совпали до последнего знака (последняя ячейка ноутбука проверяет это автоматически), в реестре появились версии 1 и 2 с одинаковым ROC-AUC 0,8488, прогнозы модели совпадают с моделью ЛР1.

10. Приложения

Репозиторий: _____ (содержимое — архив `lab2_terekhin.zip`: ноутбук, `src/`, `tests/`, `locustfile.py`, `mlruns/`, `reports/` со скриншотами, `requirements.txt`, README).

ИИ-ассистенты: Claude (Anthropic) использовался для разбора методических указаний, написания кода модулей, тестов и ноутбука, подбора совместимых версий библиотек, оформления отчёта. Результаты (метрики, задержки, отчёт Evidently) получены выполнением ноутбука и проверены по его выводу.

Ответы на контрольные вопросы (кратко).

1. Воспроизводимость — возможность повторно получить результат: фиксируются код, данные, окружение и зёрна; в Colab проверяется сбросом среды и «Выполнить всё».
2. Реестр хранит версии с метаданными (порог, данные, метрики) и псевдонимами: сервис загружает «текущую рабочую» модель по имени, замена и откат — смена псевдонима без правки кода.
3. Pickle хранит ссылку на функцию по имени модуля; функция из ноутбука (`__main__`) не найдётся в процессе сервиса.
4. 422 — ошибка клиента (неверный вход, исправить запрос), 500 — отказ сервиса; их смешение ломает мониторинг и повторные попытки.
5. Распределение задержек асимметрично, пользователь страдает от хвоста; среднее скрывает редкие долгие ответы (у нас уже при 1 пользователе p99 в 1,8 раза выше p50).
6. Когда накладные расходы на запрос велики относительно расчёта на объект и ответ не нужен немедленно — у нас ×540.
7. Дрейф данных — меняются распределения входов, виден сразу; дрейф концепта — меняется зависимость цели от входов, виден только после появления меток.
8. Указать измеримое условие с порогами, период проверки и действие, как в разделе 7.
9. Загрузка занимает секунды (реестр, десериализация), при каждом запросе она умножила бы задержку и нагрузку на диск.